- *Both an HTML web application and an API*—It's also possible to have applications that serve both needs, which can let you cater to a wider range of clients while sharing logic in your application.

In this chapter you'll learn how ASP.NET Core uses Razor Pages to handle the second of these options: creating server-side rendered HTML pages. We'll get started quickly, using a template to create a simple Razor Pages application and comparing the features of a Razor Pages app with the minimal API apps you've seen so far. In section 13.2 we look at a more complex example of a Razor Page.

Next, we take a step back in section 13.3 to look at the MVC design pattern. I discuss some of the benefits of using this pattern, and you'll learn why it's been adopted by so many web frameworks as a model for building maintainable applications.

In section 13.4 you'll learn how the MVC design pattern applies to ASP.NET Core. The MVC pattern is a broad concept that can be applied in a variety of situations, but the use case in ASP.NET Core is specifically as a UI abstraction. You'll see how Razor Pages implements the MVC design pattern and builds on top of the ASP.NET Core MVC framework.

In this chapter I'll try to prepare you for each of the upcoming topics, but you may find that some of the behavior feels a bit like magic at this stage. Try not to become too concerned about exactly how all the Razor Pages pieces tie together yet; focus on the specific concepts being addressed and how they tie into concepts you've already met. We'll start by creating a Razor Pages app to explore.

## *13.1 Your first Razor Pages application*